



F107 – Foundations of Computer Science

Lecture 6

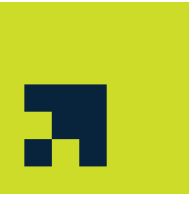
Dr. Sanjit Kumar Saha

Topics



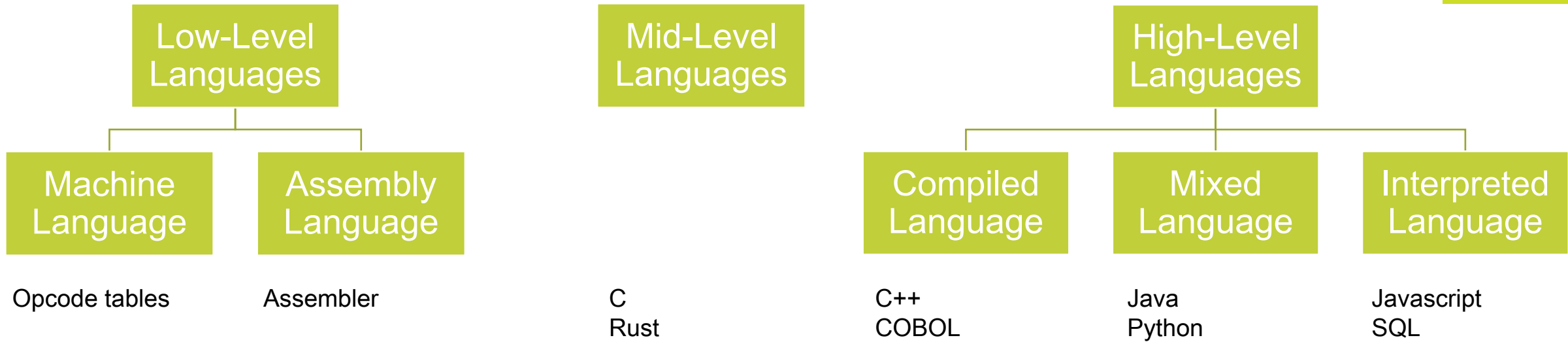
- Programming Languages
 - Types of Programming Languages
 - Types of Execution
 - Compiler
 - Interpreter
 - Programming Paradigms
 - Programming Foundations
 - Pseudocode
- Software Engineering
 - Workflow
 - Tools
- Book Chapters 9, 10, and Appendix C

Definitions

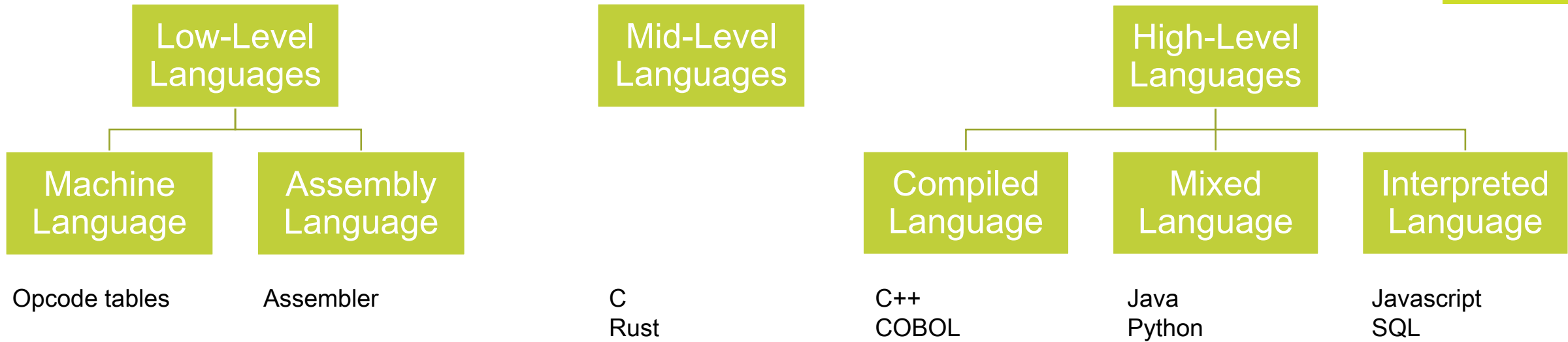


- Code
 - Logical representation of a computer program
 - Source code
 - Machine code
 - Byte code
- Programming Language (PL)
 - Special notation style to describe tasks/instructions in a computer program
 - Defined in terms of syntax and semantics
- Translation
 - Process of transformation of source code into machine code
 - 1. Compiler
 - Toolchain to translate entire source code at once into machine code
 - 2. Interpreter
 - Toolchain to translate and execute source code line-by-line

Overview of PLs



Overview of PLs



LLL Advantages	HLL Advantages
Extremely fast execution	Abstraction of HW features
No translation	Portability
Code optimization	Easy to use and debug
	Easy to learn

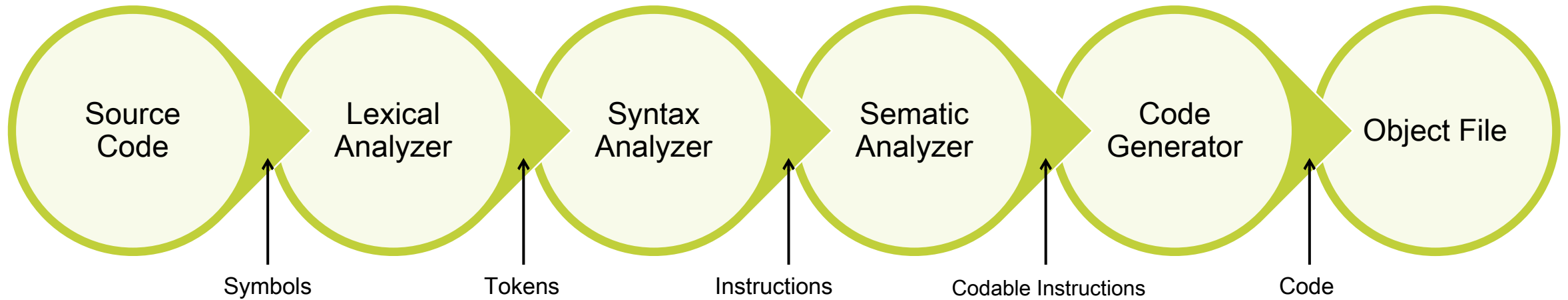
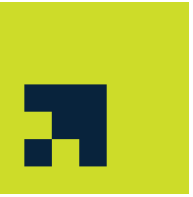
Programming Paradigms



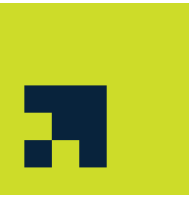
Procedural (Imperative)	Object-Oriented	Functional	Declarative
• C • FORTRAN • Pascal • Ada	• C++, C# • Java • Visual Basic	• Lisp • Scheme	• Prolog • SQL

- Program flow driven by
 - Imperative: Procedures that operate on data items
 - Object-oriented: Objects and their inherent methods, as defined by their class
 - Functional: Mathematical functions, avoiding mutable data
 - Declarative: Logical reasoning and the structure of the answer, instead of its code

Code Translation Process



Programming Foundations



Examples use C

- Identifiers
 - Names/symbols for programming entities
 - Some names are reserved
- Data types
 - Defines the range of values it can hold and the possible operations
 - Often also implies the way it is stored, as well as its storage size
 - Simple types (scalars)
 - `boolean`, `char`, `string`, `int`, `float`, ...
 - Composite types (records)
 - `array`, `list`, `map`, ...
 - `struct`, `union`, `object`, ...

```
// example
struct address {
    char *name;
    bool isLocal;
    int zipCode;
};
```

Programming Foundations



- Variables

- Modifiable memory locations
- Name used instead of address of the location
- Declaration and initialization

```
int number; // careful!  
  
int initNumber = 0;
```

- Literals

- Specify certain values in code (often data type is just implied)
- Often used in expressions or when initializing variables

- Constants

- Specific construct to declare a certain unchangeable variable

```
const int value = 19;
```

- Comments

- Descriptions that enhance understanding of code

Programming Foundations



- Expressions

- Sequence of operands and operators

```
3 + (4 * 5) >> 2
```

- Operators

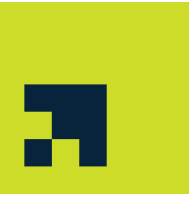
- Arithmetic operators: `+, -, *, /, %, ++, --`
- Shift operators: `>>, <<`
- Relational operators: `>, >=, <, <=, ==, !=`
- Logical operators: `!, &, &&, |, ||, ^`

- Statements

- Performs an action
 - Executes function (w/ or w/o return value)
 - Assigns value

```
y = myFunc(x);  
otherFunc();
```

Programming Foundations



- Compound Statement
 - Group of statements
 - Local variables
- Control Statements
 - Change the flow in the program
 - Sequence
 - Instructions are executed sequentially one after the other (top to bottom)
 - Selection
 - Conditionally execute instructions

case
 - Repetition (Iteration)
 - Repeat instructions in certain way
 - Exception Handling
 - Handle errors in graceful way
 - Jump
 - Move the program execution to a completely different instruction

goto

if, if-else, switch-

for, while, do-while

try-catch-finally, pass

return, break, continue
exit, throw, raise,

Programming Foundations



- If, If-Else

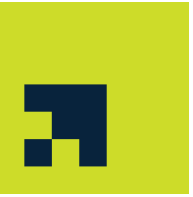
```
if (condition)
    statement1
```

```
if (condition2)
    statement2
else
    statement3
```

- Switch-case

```
switch (condition)
    case 1:
        statement1
        break
    case 2:
        statement2
        break
    default:
        statement3
```

Programming Foundations



- For

```
for (i = 0; i < MAX; i++)  
    statement
```

```
for (i = 0; i < MAX; i++)  
    if (condition1)  
        continue  
    if (condition2)  
        break  
    statement1
```

- While, do-while

```
while (condition)  
    statement
```

```
do  
    statement  
while(condition)
```

Programming Foundations



- Try-Catch-Finally

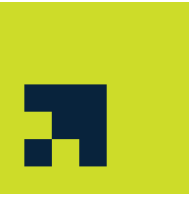
```
try
    riskyStatement
catch
    cleanUpStatement
finally
    alwaysStatement
```

- Return, goto

```
statement1
goto lab1
return

label: lab1
statement2
return
```

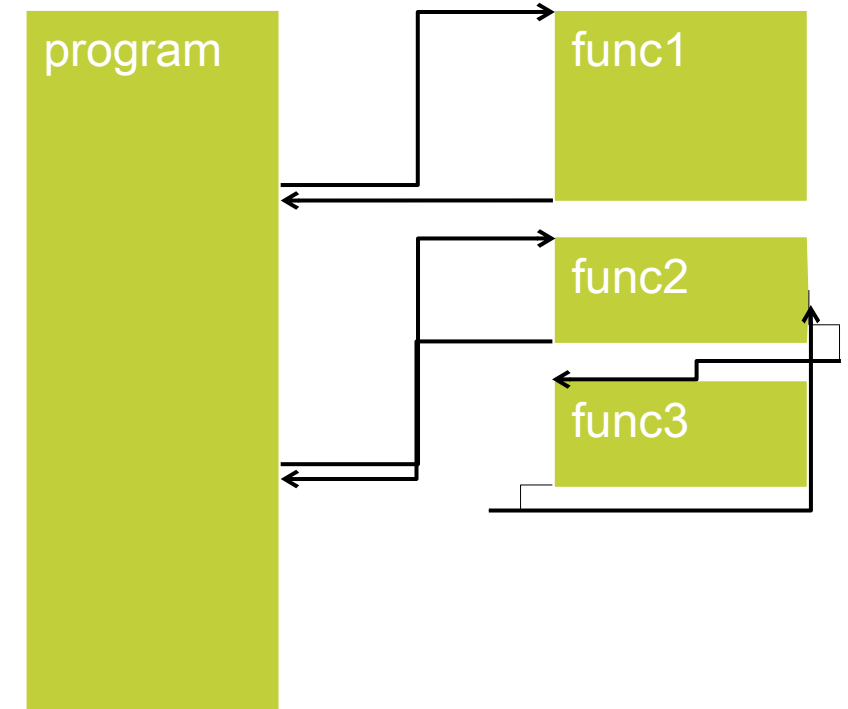
Activity



- Which of the mentioned control statements would you use to
 - terminate a program gracefully?
 - terminate a program with an exception?
 - execute a function if a certain logical statement is true?
 - process a data set multiple times?
 - process a data set multiple times, but require at least one processing?
 - execute a task infinitely often?
 - execute different functions depending on a condition with various cases?
 - process a data set multiple times, but stop early if certain condition is met?

Programming Foundations

- Sub-programs
 - Methods, functions, procedures, tasks, tasklets
 - Partial functionality is moved to own mini-program
 - Reusable
 - Modifiable
 - Exchangeable
 - Local variable(s)
 - Return value(s)
 - Parameter(s)



Activity

- Implement a square function and a swap function!

But wait! We cannot
program (yet)!



Pseudocode



- Mix of programming language and natural language
 - Abstracts away from programming language syntax
 - Focus on the algorithm (logical flow, data manipulation, ...)

```
Name:    minimumInList
Purpose:
Input:
Return:
```

Pseudocode



- Mix of programming language and natural language
 - Abstracts away from programming language syntax
 - Focus on the algorithm (logical flow, data manipulation, ...)

```
Name:    minimumInList
Purpose: Find smallest value
Input:
Return:
```

Pseudocode



- Mix of programming language and natural language
 - Abstracts away from programming language syntax
 - Focus on the algorithm (logical flow, data manipulation, ...)

```
Name:    minimumInList
Purpose: Find smallest value
Input:   L - List of integer values
Return:
```

Pseudocode



- Mix of programming language and natural language
 - Abstracts away from programming language syntax
 - Focus on the algorithm (logical flow, data manipulation, ...)

```
Name:    minimumInList
Purpose: Find smallest value
Input:   L - List of integer values
Return:  Smallest integer value in list
```

Pseudocode



- Mix of programming language and natural language
 - Abstracts away from programming language syntax
 - Focus on the algorithm (logical flow, data manipulation, ...)

```
Name:    minimumInList
Purpose: Find smallest value
Input:   L - List of integer values
Return:  Smallest integer value in list
```

```
1 var smallest = first of L
2
3
4
5 ...
6
7
8
9 return smallest
```

Pseudocode

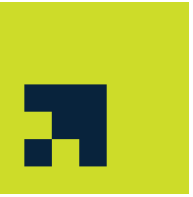


- Mix of programming language and natural language
 - Abstracts away from programming language syntax
 - Focus on the algorithm (logical flow, data manipulation, ...)

```
Name:    minimumInList
Purpose: Find smallest value
Input:   L - List of integer values
Return:  Smallest integer value in list
```

```
1 var smallest = first of L
2
3 while (not end of list) {
4     if (next of L < smallest) {
5         smallest = next of L
6     }
7 }
8
9 return smallest
```

Pseudocode



- Mix of programming language and natural language
 - Abstracts away from programming language syntax
 - Focus on the algorithm (logical flow, data manipulation, ...)

```
Name:    minimumInList
Purpose: Find smallest value
Input:   L - List of integer values
Return:  Smallest integer value in list
```

```
1 var smallest = first of L
2
3 while (not end of list) {
4     if (next of L < smallest) {
5         smallest = next of L
6     }
7 }
8
9 return smallest
```

Notes/Conventions:

- We use “=” for assignment, and “==” for equality check
- If you do not like { }, be precise with the indents
- You can use just “var” for variable, or specify more precise data type if needed

Activity



- Implement a square function and a swap function in pseudocode!

Task 1



- How often is `statement` executed in the following code excerpts?

1.

```
A = 5
while (A < 8) {
    statement
    A = A + 2
}
```

2.

```
for (i = 3; i < 20; i++) {
    statement
    i = i + 1
}
```

3.

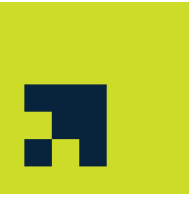
Task 2



- Change the following code segment to be a switch-case statement.

```
if (a == 1) {  
    statement1  
} else if (a == 2) {  
    statement2  
} else if (a == 5) {  
    statement3  
} else if (a >= 6 && a <= 8) {  
    statement4  
} else {  
    statement5  
}
```

Task 3



- What is the value of `A` at the end of the code execution?

```
integer A = 3
integer B = 0

A = 5 + 2*A + B
while (true) {
    function1(A)           // see →
    B = B + 1
    if (B == A) {
        A = 2 * A
        break
    }
}
A = 2*A - (A >> 1)
```

```
// function1
integer function1(integer x) {
    x = x + 2
    if (x > 5) {
        x = x / 2
    }
    return x
}
```

Task 4

- Write a generic division function in pseudocode that:
 - Takes both `real` operands as input
 - Takes a reference for the `real` result as input
 - Gives “SUCCESS” or “ERROR” as output, depending on condition



Remember the
header

Task 5



- Write a simple requirement specification for a calculator app.

Task 6



- Provide a simple “UML-inspired” activity diagram for a calculator app.

Task 7



- Create a simple “UML-inspired” class diagram for a (part of a) calculator app.

Task 8



- Describe testing scenarios for the calculator app.
 - Ensure to describe inputs and expected outputs

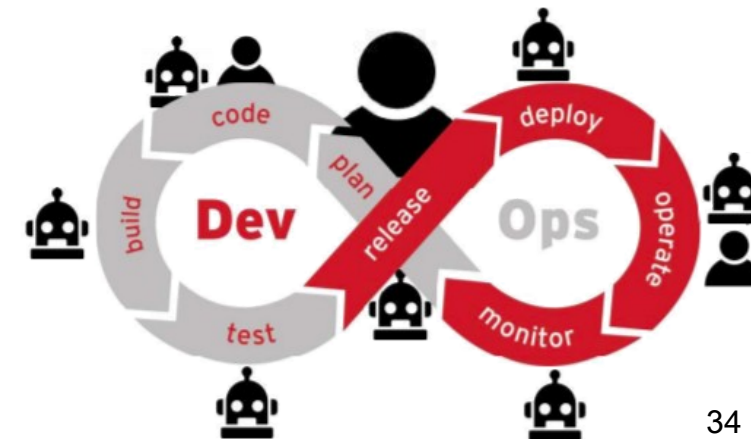
Software Engineering



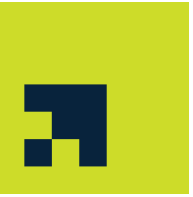
- Software creation undergoes several stages



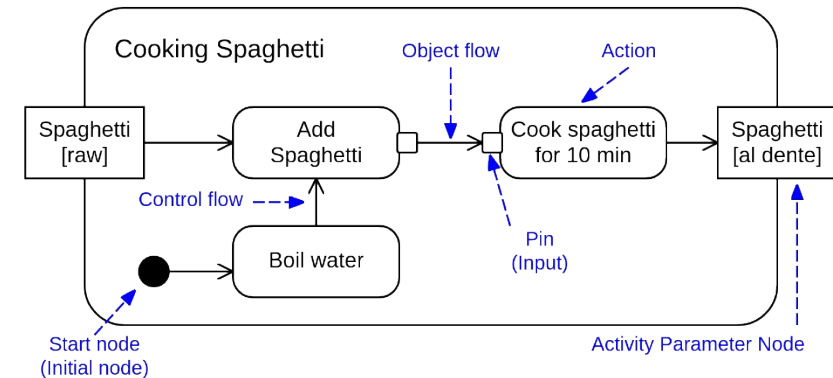
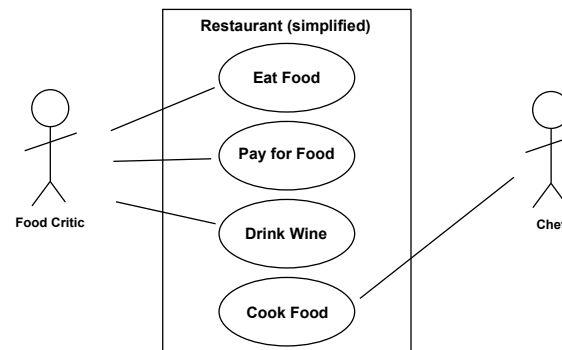
- Waterfall model: once through the phases
 - Incremental model: multiple iterations
 - V-model: design is split up and every design phase has a corresponding test phase
 - Agile models: rapid prototyping, sprints, ...
-
- Recent trend: Combination of development and operations
 - Tight coupling
 - Frequent feedback allows for improvements
 - CI/CD: Continuous Integration, Continuous Delivery
-
- Unified Modeling Language (UML) used extensively



Requirement Phase



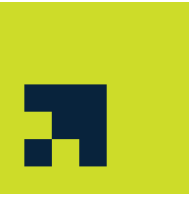
- Collection and analysis of requirements
 - Internal and external stakeholders
 - Market trends
- Final output: Requirement specification and functional specification
 - List of “must” and “must not”, “should”, and “could”
- Typical UML diagrams
 - Use-case diagram
 - Activity diagram
 - Class diagram (high level)



- [illegible]



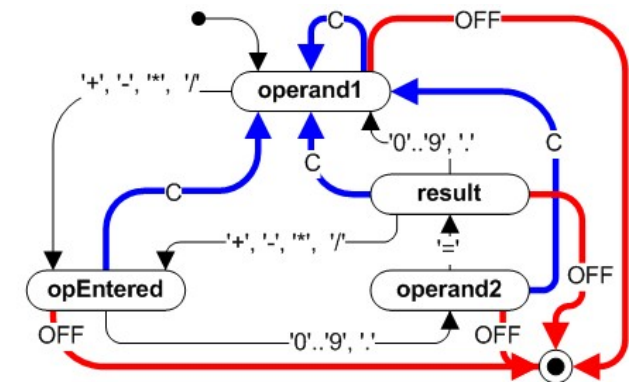
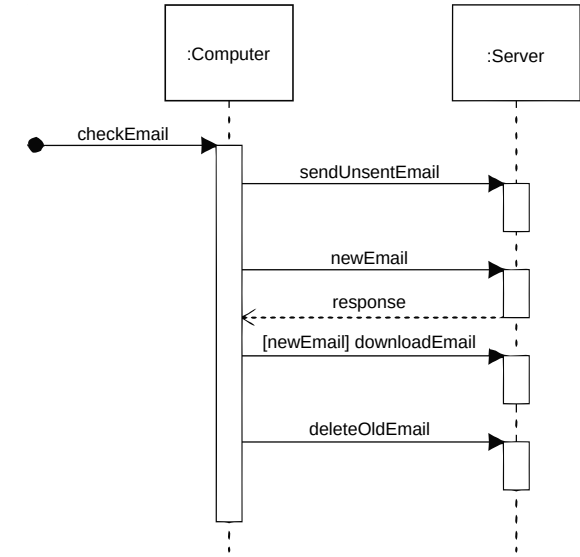
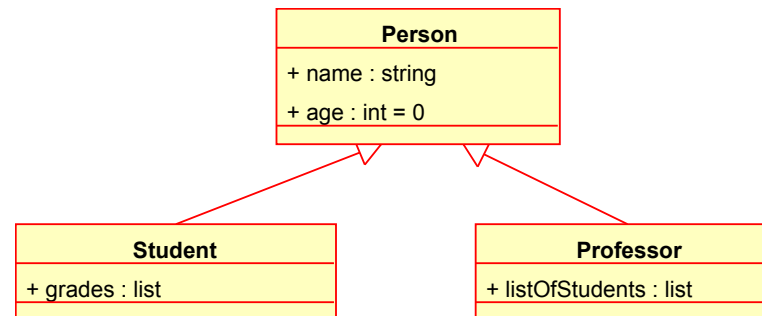
Implementation Phase



- Actual realization of the software
 - Distributed across teams, maybe even partially outsourced
- Software quality defined by
 - Operability
 - Accuracy, efficiency, reliability, security, usability
 - Maintainability
 - Changeability, correctability, flexibility, testability
 - Transferability
 - Reusability, interoperability, portability

- Typical UML diagrams

- Class diagram
- State diagram
- Sequence diagram



Testing Phase



- Validate implementation w.r.t. requirements
 - Identify differences and fix them
- Testing approaches
 - White-box vs. Black-box testing
 - Exhaustive vs. Controlled vs. Randomized Testing
 - Unit tests vs. Component tests vs. Integration tests vs. Performance/Acceptance tests
- Typical UML diagrams
 - Activity diagram
 - Sequence diagram
 - State diagram

Release Phase



- Longest phase of all five
 - Release the Software (with digital signatures)
 - Documentation
 - User documentation (feature descriptions, tutorials, ...) for externals
 - Technical documentation (installation, changelogs, ...) for externals
 - System documentation (UML diagrams, descriptions, code comments, ...) for internals
 - Process documentation (feature requests, bug reports, version control commit history, ...)
- Typical UML diagrams
 - Deployment diagram
 - Use-case diagram
 - Sequence diagram